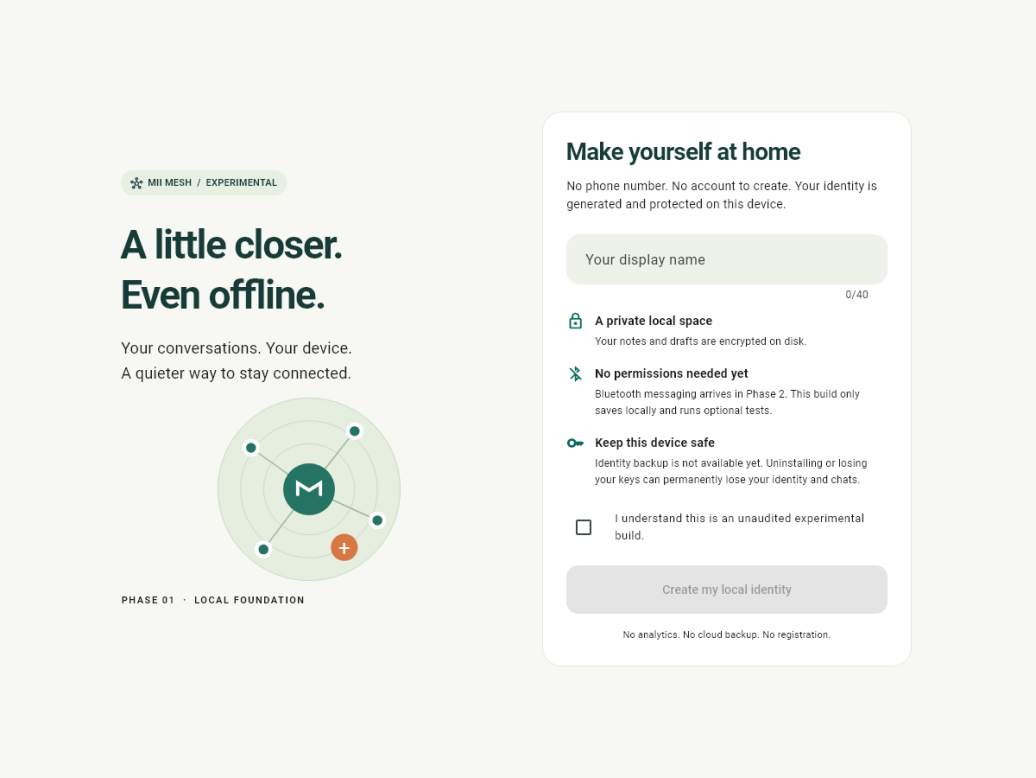
MII MESH

An Offline-First Encrypted Messaging Application
Built on a Bluetooth Low Energy Mesh Network



Mii Mesh onboarding — an identity is generated on the device itself; no account, phone number or server is involved.

Project title	Mii Mesh — offline-first encrypted messaging over a Bluetooth LE mesh
Application type	Cross-platform mobile application (Android-first)
Framework	Flutter 3.47.0 stable / Dart 3.13.0
Implementation size	7,870 lines of Dart (25 files) and 468 lines of Kotlin (4 files)
Core libraries	libsodium (sodium 4.1.0), Drift 2.34.4 + SQLite, Riverpod 3.4.3, GoRouter 18.0.1
Cryptography	Ed25519 signatures, X25519 sealed boxes, XChaCha20-Poly1305-IETF AEAD
Database	Whole-file encrypted SQLite, schema version 2
Automated tests	20 of 20 passing (verified 9 September 2026)
Status	Experimental and unaudited research prototype

Table of Contents

List of Figures

1. Abstract

Mainstream messaging applications depend entirely on centralised infrastructure. A message sent between two people standing in the same room is routed through a distant data centre, making the conversation dependent on cellular coverage, an internet subscription and the continued cooperation of a service provider. During natural disasters, network shutdowns, crowded gatherings and in remote areas, that infrastructure is exactly what becomes unavailable.

Mii Mesh removes this dependency. It forms a peer-to-peer network directly between nearby phones using **Bluetooth Low Energy**. Devices discover one another automatically and messages travel either directly to a neighbour or hop-by-hop through intermediate devices acting as relays, so two people out of Bluetooth range of each other can still communicate provided a chain of participants connects them. No server, SIM card, internet connection, phone number or account registration is used at any stage.

Confidentiality is enforced cryptographically rather than by trusting relays. Each installation generates an **Ed25519** identity on the device. Private messages are sealed to the recipient's **X25519** public key, so an intermediate device can forward a packet while remaining mathematically unable to read it. Every packet is signed, so a relay cannot alter one undetected. All local data — messages, drafts and identity material — resides in a whole-file encrypted SQLite database.

The system implements **store-and-forward** delivery: when a recipient is temporarily unreachable the encrypted message is retained and delivered automatically once a route reappears, with no action from the sender. A bounded controlled-flooding algorithm with a seven-hop limit, duplicate suppression, per-link cooldowns and message expiry prevents the broadcast storms and routing loops that naive flooding produces.

The implementation comprises a Dart protocol, router and application layer together with a native Android radio that operates simultaneously as a BLE peripheral and central, giving genuine multi-peer connectivity with chunked framing for packets larger than one BLE write. A three-section interface presents this network in plain language. All twenty automated tests pass and the Android package builds. The principal limitation is that multi-hop relaying has not yet been validated across three or more physical handsets, and voice calling is not implemented.

2. Introduction

2.1 Background and Motivation

Communication infrastructure is a single point of failure. Cellular networks fail in three broad ways: *physical failure*, when earthquakes, floods or storms destroy towers; *capacity failure*, when festivals, stadiums or demonstrations saturate a cell with thousands of devices; and *deliberate shutdown*, when connectivity is switched off by order. In each case the phones themselves remain fully functional computers with working short-range radios, yet conventional messaging applications become useless.

Mii Mesh is built on the observation that a modern smartphone already carries everything required for local communication: a Bluetooth Low Energy radio able to advertise and scan, ample local storage, and a processor fast enough for public-key cryptography. What is missing is software that treats the handset as a self-sufficient network node rather than as a client of a remote server.

2.2 Problem Statement

To design and implement a mobile messaging application that delivers end-to-end encrypted messages between nearby devices with no server, internet connection or user account; that extends usable range beyond a single Bluetooth link through multi-hop relaying; and that tolerates the intermittent connectivity inherent in a network of moving people.

2.3 Objectives

- Generate a cryptographic identity entirely on-device, with no registration or personally identifying information.
- Discover nearby participants automatically over BLE, without manual pairing.
- Encrypt messages end-to-end so that relays forward ciphertext they cannot read.
- Extend range through bounded multi-hop relaying with loop and duplicate protection.
- Retain undeliverable messages and deliver them automatically when a route reappears.
- Encrypt all data at rest so that losing the device does not expose conversations.
- Present the network in ordinary language, keeping networking terminology out of the normal interface.

2.4 Scope

Status disclosure. The protocol, router, cryptography, persistence, application layer, native dual-role BLE radio and user interface are implemented; all twenty automated tests pass and the Android package builds. A real authenticated Bluetooth exchange has been demonstrated between a phone and a laptop companion, and the application has been validated on a physical Android handset. **Multi-hop relaying between three or more phones has not yet been tested on hardware, and voice calling is not implemented.** This is an experimental prototype that has not undergone an independent security audit.

3. Literature Review

3.1 Delay-Tolerant and Opportunistic Networking

The theoretical foundation is **Delay-Tolerant Networking** (RFC 4838 [1], with the Bundle Protocol in RFC 5050 [2]), which abandons the assumption — built into TCP/IP — that a continuous end-to-end path exists between sender and receiver. It instead uses a *store-carry-forward* model: a node accepts custody of a message, stores it persistently, physically carries it as its user moves, and forwards it on encountering a suitable next hop. Mii Mesh adopts this directly; its persistent packet store is a custody store and its periodic flush is the forwarding opportunity.

Epidemic Routing (Vahdat and Becker [3]) is the simplest such strategy: forward every message to every peer encountered. It maximises delivery probability but consumes unbounded bandwidth and storage, so practical systems bound it. Mii Mesh implements *bounded controlled flooding* — epidemic in spirit, constrained by a hop limit, a duplicate-suppression cache, per-link retransmission cooldowns and message expiry.

Spray and Wait (Spyropoulos et al. [4]) restricts epidemic routing further by allocating a fixed number of copies per message, trading delivery latency for far lower overhead. It is recorded here as a candidate refinement of the routing layer.

3.2 Existing Mesh Messaging Systems

System	Transport	Characteristics and relevance to this project
FireChat (2014)	BLE and Wi-Fi Direct	The first widely adopted off-grid messenger, used during the 2014 Hong Kong protests. It proved real-world demand, but public messages were unencrypted — a limitation this project explicitly avoids.
Briar [12]	Bluetooth, Wi-Fi, Tor	A strong security model with forward secrecy and a Tor-based synchronisation protocol, but it requires explicit contact pairing, whereas Mii Mesh targets zero-configuration discovery.
Bridgefy [5]	BLE mesh	A commercial BLE mesh SDK. Albrecht et al. [5] reported serious weaknesses including absent authentication and no forward secrecy, directly motivating the signed-packet and sealed-box design adopted here.
BitChat (2025)	BLE mesh	A recent decentralised BLE mesh chat using ephemeral identities and store-and-forward; the closest architectural relative. Mii Mesh is an independent implementation and copies no source or assets.
Meshtastic [13]	LoRa radio	Achieves kilometre-scale range but requires dedicated hardware. Mii Mesh deliberately targets unmodified consumer handsets.

3.3 Bluetooth Low Energy as a Mesh Transport

The official **Bluetooth Mesh Profile** [6] uses managed flooding over the advertising bearer and was designed for building automation — many low-rate lighting and sensor nodes. It suits smartphone messaging poorly, because mobile operating systems restrict direct access to the advertising bearer and impose aggressive background execution limits. Phone-based systems, including this one, therefore implement an application-layer mesh over standard GATT connections rather than adopting the SIG profile.

Practical constraints reported in the literature and confronted directly in this implementation include a limited number of concurrent GATT connections per device, a default ATT MTU of 23 bytes that forces application-level fragmentation, platform background-execution limits requiring a foreground service, and the significant energy cost of continuous scanning.

3.4 Cryptographic Foundations

- **Ed25519** (Bernstein et al. [7]; RFC 8032 [8]) — fast, misuse-resistant signatures, used for device identity and packet authenticity.
- **X25519 sealed boxes** — anonymous public-key encryption to a recipient's key, allowing relays to forward messages they cannot read.
- **XChaCha20-Poly1305-IETF** [9], [10] — authenticated encryption with a 192-bit nonce, making random nonce generation safe without a counter; used for the direct Bluetooth link and the local test transport.
- **The Noise Protocol Framework** [11] — the standard basis for authenticated key exchange with forward secrecy. Not implemented here, and identified as a release-blocking requirement.

4. System Modules

The system is organised into eight modules with a strict boundary between interface, application logic and networking. Cryptography, persistence and transport all sit outside the widget layer: interface code calls an application service and never touches a key or a radio directly.

#	Module	Responsibility	Status
1	Identity & Cryptography core/crypto/vault.dart	Generates and protects the Ed25519 identity seed and X25519 mesh key in platform secure storage; derives the database key; renders a readable fingerprint.	Implemented, tested
2	Encrypted Persistence core/database/	Typed Drift/SQLite schema v2 for profile, conversations, messages, outbox, preferences, mesh peers and stored packets, with mandatory whole-file encryption.	Implemented, tested
3	Mesh Protocol transports/mesh/mesh_protocol.dart	Signed packet format, sealed private payloads, the four message kinds, and a hardened bounded parser rejecting nesting and oversized input.	Implemented, tested
4	Mesh Router transports/mesh/mesh_router.dart	Bounded controlled flooding, duplicate suppression, TTL enforcement, peer table maintenance, acknowledgements and store-and-forward flushing.	Implemented, tested
5	Mesh Service transports/mesh/mesh_service.dart	Binds the router to the native radio and the database: link events, periodic announcement and flush, durable queueing, optimistic send and lifecycle.	Implemented, tested
6	Native BLE Radio android/…/MeshRadio.kt	Simultaneous GATT server and client, service-filtered advertising and scanning, per-device links, chunked framing and a foreground service.	Implemented, compiles, not hardware-tested
7	Messaging Service features/chats/chat_service.dart	Conversation and message lifecycle, persistent outbox, bounded exponential backoff, retry and cancel, drafts and deduplication.	Implemented, tested
8	User Interface app/, features/	Three-section navigation, nearby list, conversations with stickers and reactions, diagnostics, onboarding, responsive and accessible layouts.	Implemented, tested

4.1 Module Interaction

```
User Interface   (Chats / Nearby / Calls)
      |   plain-language state; never touches keys or the radio
      v
Chat Service  <-------->  Encrypted SQLite  (messages, outbox, peers,
      |                                      stored packets, drafts)
      v                          ^
Mesh Service   -------- persist before forward or acknowledge
      |   periodic announce (15 s) and flush (5 s)
      v
Mesh Router    <------  Identity & Crypto   (sign / seal / open)
      |   duplicate cache, TTL, peer table, pending custody queue
      v   MethodChannel 'mii/mesh'
Native BLE Radio (Kotlin)
      |   GATT server (advertise) + GATT client (scan/connect)
      v
BLE radio   <== over the air ==>   neighbouring devices
```

4.2 Message Types

The protocol defines a compact set of message kinds so the networking layer can decide how each payload travels. Reactions and stickers are deliberately tiny: a reaction references its target message by identifier and carries a single emoji, and a sticker transmits a catalogue identifier rather than image data, so no picture is ever sent over the radio.

Kind	Payload	Transport treatment
presence	Display name and X25519 public key	Broadcast to every link; valid for 60 seconds
text	UTF-8 body, up to 2048 bytes	Sealed to the recipient; relayed and held in custody
sticker	Identifier from a six-entry catalogue	Sealed; identifier only, never image bytes
reaction	Emoji plus target message identifier	Sealed; minimal size, original never resent
ack	Target message identifier	Sealed; clears the sender's pending queue

5. Software and Hardware Requirements

5.1 Development Environment

Component	Version or specification
Framework	Flutter 3.47.0 (stable channel)
Language	Dart 3.13.0; Kotlin for the Android radio
Android compile SDK	37
Android NDK	28.2.13676358
JDK	21
IDE	Android Studio / IntelliJ or VS Code with the Flutter plugin
Host operating system	Windows 11 Pro (build 26200)
Version control	Git

5.2 Key Dependencies

Package	Version	Purpose
sodium	4.1.0+1	libsodium bindings — Ed25519, X25519 sealed boxes, XChaCha20-Poly1305
drift, sqlite3	2.34.4, 3.5.2	Typed, whole-file encrypted local database
flutter_secure_storage	11.0.0	Platform-protected key storage (Keystore, Keychain)
flutter_riverpod	3.4.3	Application state management and dependency injection
go_router	18.0.1	Declarative navigation
freezed, json, cbor	3.1.0, 4.12.0, 6.5.1	Immutable models and a bounded envelope codec
uuid	4.6.0	UUID message identifiers used for duplicate detection

5.3 Hardware Requirements

Minimum target device

Requirement	Specification
Operating system	Android 8.0 (API 26) or later; iOS 12+ scaffolded but unvalidated
Bluetooth	Bluetooth 4.0+ with BLE, supporting both central and peripheral roles
RAM	2 GB minimum, 4 GB recommended
Storage	Approximately 100 MB for the application and its local database
Processor	64-bit ARM (arm64-v8a)

The peripheral role is essential rather than optional. A device that can only scan is able to discover others but cannot itself be discovered, and therefore cannot serve as a relay. Most Android handsets released from 2015 onward support both roles concurrently.

Hardware used for validation

- **Samsung SM-M558B (Android 16)** — verified installation, launch, and recovery of saved notes and drafts after a force-stop.
- **Windows laptop companion** — a Python BLE companion used to verify a real, authenticated single-hop Bluetooth message round trip.
- **Not yet available:** the three or more handsets needed to validate multi-hop relaying, route repair and store-and-forward across genuine disconnections.

5.4 Android Permissions

Capability	Permissions
Bluetooth (Android 12+)	BLUETOOTH_SCAN, BLUETOOTH_CONNECT, BLUETOOTH_ADVERTISE
Bluetooth (legacy)	BLUETOOTH, BLUETOOTH_ADMIN through API 30
Legacy discovery	ACCESS_COARSE_LOCATION, ACCESS_FINE_LOCATION
Alerts	POST_NOTIFICATIONS, VIBRATE
Background operation	FOREGROUND_SERVICE with CONNECTED_DEVICE type, WAKE_LOCK

Scan results are declared `neverForLocation`, so nearby-device discovery is not used to derive the user's physical location. Permission is requested at the moment the feature is first used, with an explanation shown beforehand, and denial is handled gracefully so that local notes remain fully usable.

6. Algorithms

6.1 Packet Structure

Every mesh packet carries a fixed, signed header. The signature covers all header fields and the payload, so a relay can neither forge nor modify a packet. The hop counter lies outside the signed region, since each relay must legitimately increment it.

```
MeshPacket {
  version        : 2
  messageId      : UUID                        (duplicate detection)
  senderId       : Ed25519 public key, base64url
  destinationId  : recipient key, or '*' for broadcast presence
  timestamp      : Unix milliseconds
  kind           : 'presence' | 'sealed'
  payload        : base64url  (sealed ciphertext, max 4096 bytes)
  maxHops        : 7
  ---- signed above this line (Ed25519 detached) ----
  signature      : base64url
  hops           : 0..7                        (incremented by each relay)
}
```

6.2 Encryption and Sealing

A private message is encrypted with an anonymous sealed box against the recipient's X25519 public key. Only the recipient's secret key can open it, so every intermediate relay handles ciphertext alone. The plaintext within the seal is itself a validated triple of kind, body and reference.

```
function sendPrivate(peer, kind, body, reference):
    validate(kind, body, reference)
    plaintext  <- encode([kind, body, reference])
    ciphertext <- crypto_box_seal(plaintext, peer.publicKey)
    packet     <- createPacket(dest    = peer.id,
                               kind    = 'sealed',
                               payload = ciphertext)
    packet.signature <- ed25519_sign(packet.header, myIdentity.secretKey)

    persist(packet)          // durable before any transmission
    showInUI(packet)         // optimistic: never wait for the radio
    for each link in activeLinks:
        write(link, packet)
```

6.3 Core Routing Algorithm — Bounded Controlled Flooding

This is the heart of the system. Each received packet is verified, then either consumed locally or relayed onward. Four independent mechanisms bound the flood: signature verification, a duplicate cache, a hop limit and packet expiry. Persistence always completes before a packet is forwarded or acknowledged, so a crash cannot silently lose a message whose receipt has already been reported to its sender.

```
function onPacketReceived(link, bytes):

    packet <- decode(bytes)                 // rejects malformed input
    if not verifySignature(packet):   discard; return
    if packet.sender == myId:         discard; return   // own echo
    if packet.expired():              discard; return

    // --- 1. Presence: learn about a peer --------------------------
    if packet.kind == 'presence':
        validate(displayName, publicKey)
        if known(sender) and known.key != publicKey:
            reject                          // pinned key changed
        if unknown or packet.hops <= known.hops:
            peers[packet.sender] <- Peer(name, key,
                                         lastSeen = now,
                                         hops     = packet.hops,
                                         via      = link)

    // --- 2. Addressed to me: consume, never relay -----------------
    if packet.destination == myId:
        content <- sealOpen(packet.payload, mySecretKey)

        if content.kind == 'ack':
            remove content.reference from pendingQueue
            mark message delivered
            return

        persist(content)                    // BEFORE acknowledging
        deliverToUI(content)
        send ack(packet.id) to sender
        return

    // --- 3. Not mine: relay under strict bounds -------------------
    if seenCache.contains(packet.id):  discard; return  // duplicate
    seenCache.insert(packet.id, expiry = packet.expires)

    if packet.hops >= MAX_HOPS (7):    discard; return  // TTL spent

    relayed <- packet.withHops(packet.hops + 1)

    if packet.kind == 'sealed':
        pendingQueue.add(relayed)           // accept custody

    for each l in activeLinks where l != link:  // never echo to source
        write(l, relayed)
        lastSent[l, packet.id] <- now
```

6.4 Store-and-Forward Delivery

Packets held in custody are retried periodically. A per-link cooldown stops a node saturating a neighbour with repeated copies of the same packet, and expired packets are dropped so that storage stays bounded.

```
function flush():                    // every 5 seconds
    for each packet in pendingQueue:
        if packet.expired():
            remove from queue and storage
            continue
        for each link in activeLinks:
            if now - lastSent[link, packet.id] < COOLDOWN (15 s):
                continue                // too soon; avoid flooding
            write(link, packet)
            lastSent[link, packet.id] <- now
```

6.5 Link-Layer Framing

A signed packet can reach roughly four kilobytes, far larger than a single BLE write at the negotiated MTU. The native radio therefore splits each packet into indexed chunks and reassembles them on receipt, rejecting out-of-order indices, implausible totals and stalled transfers.

```
send(packet):
    chunks <- split(packet, mtuPayloadSize)
    for i in 0 .. chunks.length-1:
        write([i, chunks.length] ++ chunks[i])

receive(frame):
    (index, total, data) <- parse(frame)
    if total not in 1..512        : reset; return  // implausible
    if index != expectedIndex     : reset; return  // out of order
    if index > 0 and stalled(10 s): reset; return  // abandoned
    buffer.append(data)
    if buffer.size > 4096         : reset; return  // bounded memory
    if index == total - 1         : deliver(buffer); reset
```

6.6 Bounds and Safety Parameters

Parameter	Value	Purpose
Maximum hops (TTL)	7	Bounds the flood radius and prevents infinite loops
Duplicate cache	4,000 identifiers	Suppresses copies arriving by multiple paths
Message lifetime	24 hours	Bounds storage; expired packets are purged
Presence validity	60 seconds	Separates reachable peers from recently-seen ones
Pending custody queue	500 packets	Bounds store-and-forward storage
Peer table	256 peers	Prevents memory exhaustion from forged identities
Per-link cooldown	15 seconds	Prevents saturating a single neighbour
Maximum payload	4,096 bytes	Bounds parser work and radio occupancy
Chunks per packet	512	Bounds reassembly buffers
Queued radio events	128	Applies backpressure to the native event stream
Retry backoff	Exponential with jitter, 5 attempts	Avoids synchronised retry storms

6.7 Reachability Classification

Internal connection state is mapped to ordinary language for the interface. GATT, MTU, RSSI and UUID terminology never appears unless the developer diagnostics screen is deliberately opened.

Internal condition	Shown to the user
hops = 0 and seen within 60 s	Nearby
hops > 0 and seen within 60 s	Via mesh
Last seen more than 60 s ago	Recently seen
Adapter disabled	Bluetooth is off
Scanning with no peers found	Searching for nearby people…
Write failed, retry pending	Connection interrupted — retrying…

7. Screenshots

The following are genuine rendered outputs of the application, captured from its own widget test harness at real device resolutions. No mock-ups or illustrations are used. Peer entries in Figures 4 and 7 are seeded test data, since populating them from live radio traffic would require several physical handsets.

7.1 Onboarding

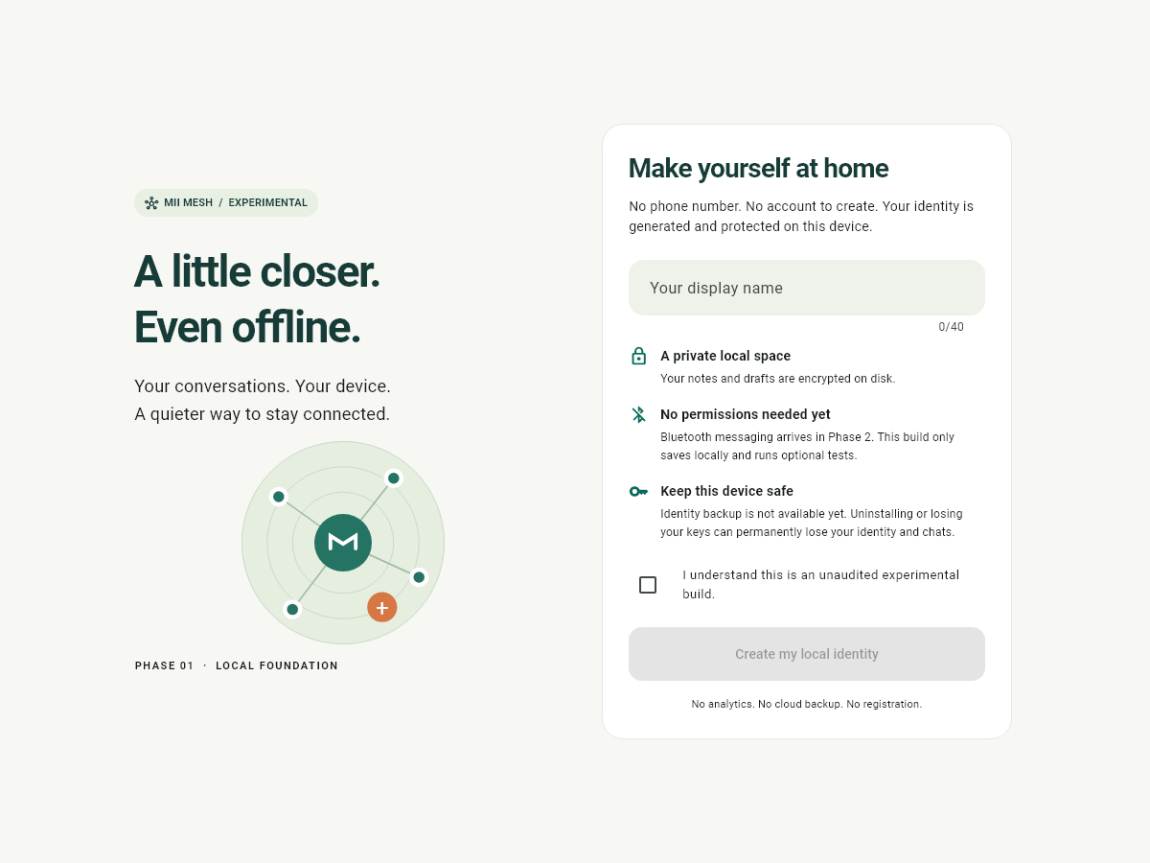


Figure 1 — Onboarding at 1200 × 900. No phone number or account is requested; the identity is generated on the device, and the user must explicitly acknowledge that the build is experimental before continuing.

Onboarding is deliberately honest rather than promotional. It states that notes and drafts are encrypted on disk, that identity backup does not yet exist, and that losing the device loses the identity. The primary action stays disabled until a display name has been entered and the acknowledgement ticked.

7.2 Chats on a Wide Layout

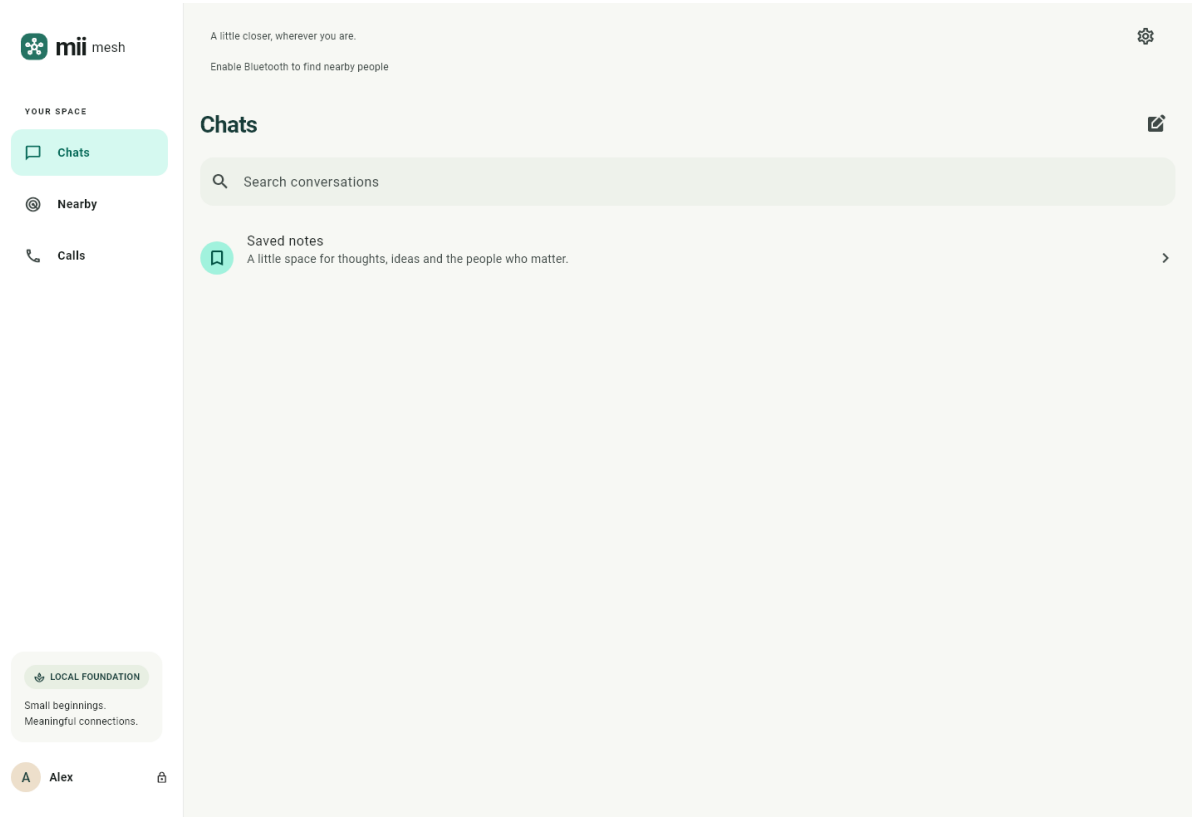


Figure 2 — Chats at 1440 × 980, showing the three-section navigation, the plain-language mesh status line, and settings reduced to a single icon.

The interface uses three sections only — **Chats**, **Nearby** and **Calls** — with settings moved to one corner icon. Beneath the title a status line reports the state of the mesh in ordinary words; here it reads *“Enable Bluetooth to find nearby people”*.

7.3 Nearby — Permission and Discovery

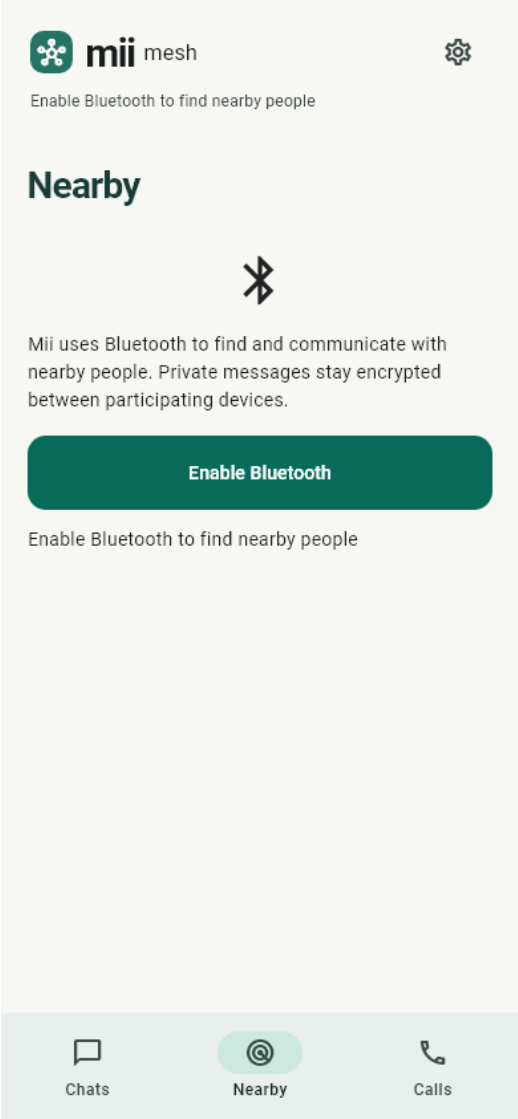


Figure 3 — First use. The purpose of Bluetooth access is explained before any system permission dialog appears.

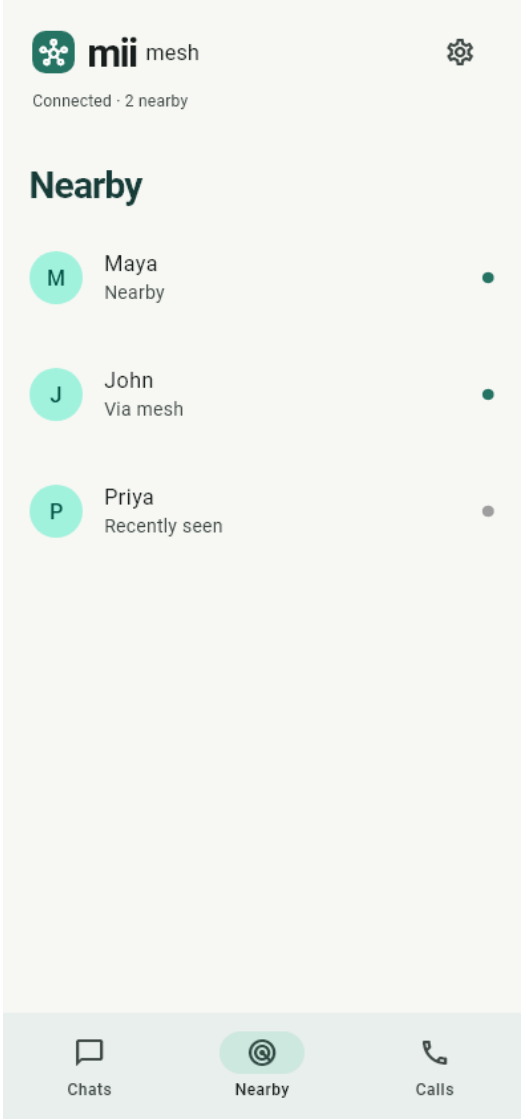


Figure 4 — Peers discovered, showing the three reachability states in plain language.

Discovery is automatic; there is no scan button to press and no manual pairing step. Each entry states how that person is reachable — **Nearby** for a direct link, **Via mesh** when the route passes through intermediate devices, and **Recently seen** once presence has lapsed. Tapping a person opens a conversation.

7.4 Conversation and Calls

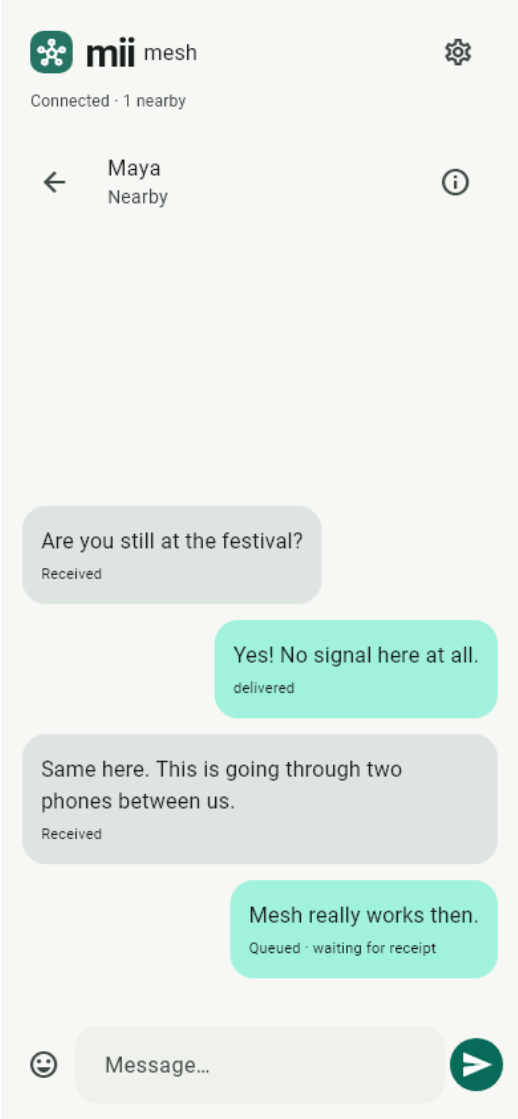


Figure 5 — A mesh conversation showing delivery states: received, delivered, and queued awaiting a receipt.

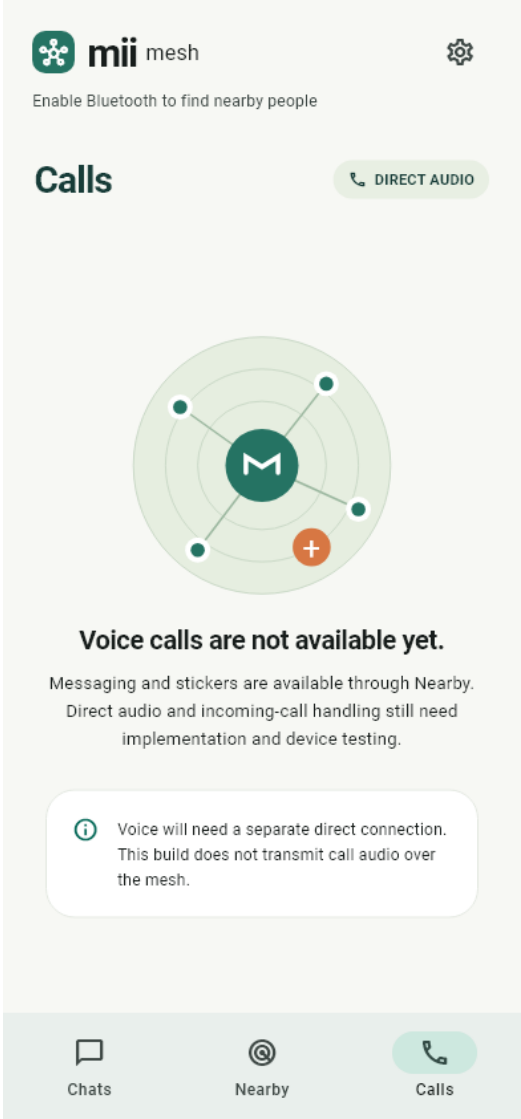


Figure 6 — The Calls section states plainly that voice calling is not implemented.

The conversation view follows familiar messaging conventions, so no learning is required. The header repeats the correspondent's reachability, each message carries its lifecycle state, and messages appear immediately on sending while encryption and routing proceed in the background. A sticker catalogue and six reactions are available from the composer; both travel as identifiers rather than images.

The Calls section illustrates a deliberate design decision. Rather than presenting a control that would fail, it states that voice calling is unavailable and explains why: continuous audio cannot be carried hop-by-hop across a BLE mesh, and a separate direct connection is required.

7.5 Diagnostics and Settings

Figure 7 — Developer diagnostics: link counts, relay and rejection counters, per-peer hop distance and last-seen time.

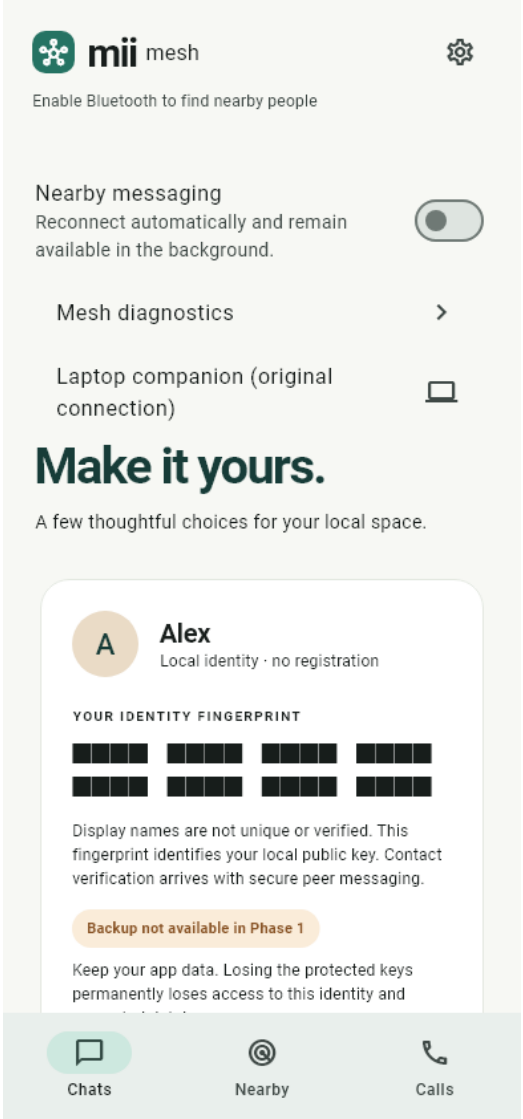


Figure 8 — Settings, including identity fingerprint, appearance and mesh controls.

All networking detail is confined to the diagnostics screen, which an ordinary user never needs to open. It exposes the number of direct links, known peers, envelopes awaiting delivery, packets relayed and packets rejected, together with each peer's hop distance and last-seen timestamp — precisely the information required to debug a mesh, and precisely the information that would confuse a normal user if shown on the main screens.

8. Testing and Validation

8.1 Automated Test Suite

The suite was executed in full on 9 September 2026. **All twenty tests passed.**

Suite	Cases	Coverage
foundation_test.dart	6	Ed25519 RFC 8032 known-answer vector; AEAD tampering and header substitution; encrypted database creation, reopening and wrong-key rejection; randomised parser fuzzing
widget_test.dart	4	Phone, tablet and large-text layouts; onboarding and conversation reference images
mesh_router_test.dart	3	Sealed text authenticity and recipient-only decryption; four-node relaying, single acknowledgement and alternate paths
queue_integration_test.dart	3	Persistence before send; five-attempt failure limit; retry, cancel and rollback
ble_packet_test.dart	2	Direct Bluetooth framing, direction and expiry authentication; rejection of invalid and nested payloads
mesh_persistence_test.dart	1	Schema v1 to v2 upgrade preserving identity and history; mesh outbox surviving restart
mesh_permissions_test.dart	1	Permission denial handled without crash or data loss

8.2 Verified Results

- Static analysis reports **no issues** across the entire source tree.
- **20 of 20 automated tests pass**, including all reference-image comparisons.
- The **Android ARM64 debug package builds successfully**, so the native Kotlin radio compiles against the Android SDK.
- The Ed25519 known-answer test matches RFC 8032 vector 1.
- Tampering with authenticated ciphertext and substituting packet headers are both correctly rejected.
- A four-node relay test confirms that opaque packets traverse intermediate nodes, are acknowledged exactly once, and still arrive when an alternate path is used.
- 1,500 randomised malformed inputs and targeted parser cases were rejected without crash or resource exhaustion.
- Encrypted database creation, reopening, wrong-key rejection and the schema v1 to v2 upgrade all pass, with identity and history preserved.
- A real authenticated Bluetooth round trip was completed between a phone and the laptop companion.
- Physical validation on a Samsung SM-M558B: installation, launch, and recovery of saved notes and drafts after force-stop.

8.3 Limitations of the Present Validation

- **Multi-hop relaying is unverified on hardware.** The four-node relay test exercises the routing logic in Dart with simulated links. It does not exercise real radios, genuine range boundaries, interference or mobility. Confirming the mesh requires three or more physical handsets, and no host-side test substitutes for that.
- **The native radio compiles but has not been hardware-validated.** Concurrent advertising and scanning behaves differently across Android vendors and must be measured on real devices.
- **Voice calling is not implemented.**
- **No forward secrecy.** Sealed boxes protect a message from relays, but compromise of an identity key would expose past messages encrypted to it. A Noise handshake is required and is release-blocking.
- **No independent security audit** has been carried out.

> **A note on reproducing these results on Windows.** Host tests require a signature-verified libsodium binary supplied through a temporary dependency override, because Windows SDK headers for a source build are absent. The override is removed after the run; Android builds the library normally from source.

9. Conclusion

This project set out to demonstrate that useful, private messaging is possible between nearby smartphones with no server, no internet connection and no user account. That has been achieved to the point where only hardware validation remains.

A complete mesh stack has been designed and implemented: a signed packet format, anonymous sealed-box encryption to the recipient's public key, a bounded controlled-flooding router with duplicate suppression, hop limiting, expiry and store-and-forward custody, a service binding that router to durable storage, and a native Android radio operating simultaneously as BLE peripheral and central with chunked framing for packets exceeding a single write. The cryptographic layer rests on libsodium primitives and is validated against published vectors. All data at rest is held in a whole-file encrypted database, and identity material is protected by platform secure storage. All twenty automated tests pass and the Android package builds.

The architectural decision that matters most is the separation of routing from confidentiality. Because private payloads are sealed to the recipient's X25519 key rather than merely link-encrypted, a relay performs its forwarding role while remaining cryptographically unable to read what it carries. This is what makes relaying by untrusted strangers acceptable at all, and it directly addresses the weaknesses that published analyses identified in earlier BLE mesh messengers.

Comparable attention went into bounding the system. Naive flooding destroys mesh networks through broadcast storms, routing loops and unbounded memory growth. Eleven independent bounds — covering hop count, duplicate history, message lifetime, custody and peer table sizes, per-link cooldown, payload and chunk limits, event backpressure and retry behaviour — constrain the algorithm so that its worst case stays predictable. The parser was hardened against hostile input and confirmed by randomised fuzzing.

On the interface side, navigation was reduced to three sections and networking vocabulary replaced with ordinary language. A user sees *Nearby*, *Via mesh* or *Recently seen*; hop counts, link states and radio detail remain in a diagnostics screen where they belong. Where a capability does not exist, the interface says so plainly rather than offering a control that would fail.

The honest boundary of this work is hardware validation. A real authenticated single-hop Bluetooth exchange has been demonstrated and the application runs and recovers correctly on a physical Android handset, but multi-hop relaying between three or more phones — the definitive test of a mesh — has not been performed. The routing algorithm is implemented, reasoned about and unit-tested against simulated links; it is not yet field-proven, and describing it otherwise would overstate the evidence.

Within that boundary the project succeeds. It shows that an ordinary smartphone can act as a self-sufficient, privacy-preserving network node using only the radio it already carries, and it provides a tested foundation on which the remaining hardware validation can proceed.

10. Future Scope

10.1 Immediate Next Steps

- **Multi-device hardware validation.** Exercise A↔B, A↔B↔C and A↔B↔C↔D topologies; remove an intermediate node and confirm an alternative route is found; verify store-and-forward across real

disconnections and physical movement.

- **Radio measurement and tuning.** Characterise concurrent advertising and scanning across Android vendors, negotiate and exploit larger MTUs, and tune connection intervals and back-off to avoid connection storms in dense groups.
- **iOS implementation.** The Dart layer is portable, but the native radio and background behaviour must be written and validated against Apple's more restrictive background execution model.

10.2 Security Hardening (Release-Blocking)

- **A Noise protocol handshake** with X25519, HKDF-SHA-256, transcript binding and session rotation, providing mutual authentication and **forward secrecy**, which the current design lacks.
- **Contact verification** by fingerprint comparison or QR exchange, with explicit warnings when a pinned key changes.
- **Persistent replay protection** across restarts, together with anti-amplification limits and per-peer quotas.
- **Identity backup and recovery,** which does not exist today — losing the device permanently loses the identity.
- **An independent security audit** and a full transitive dependency and licence review.

10.3 Feature Expansion

- **Voice communication.** Call signalling can travel over the mesh, but continuous audio must not. The intended design negotiates a call over Bluetooth and then establishes a direct high-bandwidth peer-to-peer link for the audio itself. Where no such link is possible, a low-bitrate push-to-talk voice message relayed through the mesh is the honest fallback.
- **Media transfer** with per-attachment random keys, chunk hashing, resumable transfer and strict size limits.
- **Encrypted group channels,** requiring membership authorisation and key rotation on every membership change.
- **Priority queues,** so that a large transfer can never delay a short text message or call signalling.
- **Adaptive power management** with active, balanced and idle duty cycles selected from battery level and application state.

10.4 Research Directions

- Smarter routing — Spray-and-Wait copy limiting or utility-based forwarding to cut redundant transmissions.
- Hybrid transports, opportunistically using Wi-Fi Direct for bulk data while BLE handles discovery and control.
- Traffic-analysis resistance through padding, cover traffic and randomised forwarding delays.
- A scalability study of flooding behaviour as participant density grows.
- Energy characterisation of duty-cycling strategies under realistic mobility.

Closing note. Mii Mesh is an experimental, unaudited research prototype. It should not be relied upon where a communication failure or a disclosure of message content would cause harm. Its value lies in demonstrating a working architecture and in documenting precisely what remains to be verified.

11. References

Standards documents are identified by their permanent RFC or specification number. Software projects referenced as design comparisons are listed by project name.

[1] Cerf, V., Burleigh, S., Hooke, A., Torgerson, L., Durst, R., Scott, K., Fall, K. and Weiss, H. *Delay-Tolerant Networking Architecture.* RFC 4838, Internet Engineering Task Force, 2007.

[2] Scott, K. and Burleigh, S. *Bundle Protocol Specification.* RFC 5050, Internet Engineering Task Force, 2007.

[3] Vahdat, A. and Becker, D. *Epidemic Routing for Partially Connected Ad Hoc Networks.* Technical Report CS-2000-06, Department of Computer Science, Duke University, 2000.

[4] Spyropoulos, T., Psounis, K. and Raghavendra, C. S. *Spray and Wait: An Efficient Routing Scheme for Intermittently Connected Mobile Networks.* Proceedings of the ACM SIGCOMM Workshop on Delay-Tolerant Networking (WDTN), 2005.

[5] Albrecht, M. R., Blasco, J., Jensen, R. B. and Mareková, L. *Mesh Messaging in Large-Scale Protests: Breaking Bridgefy.* Topics in Cryptology — CT-RSA, Lecture Notes in Computer Science, Springer, 2021.

[6] Bluetooth Special Interest Group. *Mesh Profile Specification, version 1.0.* 2017.

[7] Bernstein, D. J., Duif, N., Lange, T., Schwabe, P. and Yang, B.-Y. *High-Speed High-Security Signatures.* Journal of Cryptographic Engineering, volume 2, number 2, 2012.

[8] Josefsson, S. and Liusvaara, I. *Edwards-Curve Digital Signature Algorithm (EdDSA).* RFC 8032, Internet Engineering Task Force, 2017.

[9] Nir, Y. and Langley, A. *ChaCha20 and Poly1305 for IETF Protocols.* RFC 8439, Internet Engineering Task Force, 2018.

[10] Arciszewski, S. *XChaCha: eXtended-nonce ChaCha and AEAD_XChaCha20_Poly1305.* Internet-Draft, Crypto Forum Research Group, IETF.

[11] Perrin, T. *The Noise Protocol Framework,* revision 34. 2018.

[12] The Briar Project. *Briar — Secure Messaging, Anywhere.* briarproject.org.

[13] Meshtastic Project. *Meshtastic — An Open Source, Off-Grid Mesh Communication Platform.* meshtastic.org.

[14] Frank Denis and contributors. *libsodium Documentation.* doc.libsodium.org.

[15] Google LLC. *Bluetooth Permissions.* Android Developers documentation, developer.android.com.

[16] Google LLC. *Foreground Service Types.* Android Developers documentation, developer.android.com.

[17] Google LLC. *Flutter Documentation.* docs.flutter.dev.

Appendix A Source Layout

The repository is organised so that cryptography, persistence and transport remain outside the widget layer. Generated files (Drift and Freezed output) are omitted from this listing.

```
lib/
  app/                      theme, router, Riverpod bootstrap, responsive shell
  core/crypto/vault.dart    identity seed, fingerprint, platform secret storage
  core/database/            typed Drift tables, mandatory file encryption (v2)
  core/protocol/            Freezed envelope and bounded CBOR codec
  features/onboarding/      consent and local identity setup
  features/chats/           conversation service and chat interface
  features/nearby/          mesh nearby list, conversation, diagnostics
  features/calls/           calls section (voice not implemented)
  features/settings/        appearance, identity, diagnostics entry
  transports/mesh/          protocol, router and mesh service
  transports/ble/           direct laptop link framing
  transports/fake/          explicit in-process simulator

android/app/src/main/kotlin/app/miimesh/mii_mesh/
  MainActivity.kt           method and event channel registration
  MeshRadio.kt              dual-role GATT server and client, chunked framing
  BlePeripheral.kt          direct laptop companion link
  MeshForegroundService.kt foreground service for background availability

test/                       crypto, persistence, router, queue, widget and
                            report-screenshot suites
docs/                       protocol, schema, threat model, permissions,
                            test results and this report
```